SWE PLACEMENT ROADMAP — Basics to Top Companies

Start Date: June 25, 2026
Target: Cracking FAANG + Top product-based companies (Amazon, Google, Microsoft, Uber, Stripe, Atlassian)
Current Status: LeetCode 53 solved | CF Unrated | GitHub empty
Daily Commitment: 5-7 hours (adjust based on college)

HOW THIS ROADMAP IS STRUCTURED

```
PHASE 0 (3 weeks)  → Foundation & STL           → Jun 25 - Jul 15
PHASE 1 (14 weeks) → Core DSA (Striver A2Z)      → Jul 16 - Oct 21
PHASE 2 (8 weeks)  → CS Fundamentals + LLD       → Oct 22 - Dec 16
PHASE 3 (8 weeks)  → System Design + Advanced    → Dec 17 - Feb 10
PHASE 4 (6 weeks)  → Projects + Resume + OS      → Feb 11 - Mar 24
PHASE 5 (14 weeks) → Placement Prep + Mocks      → Mar 25 - Jun 30
PHASE 6 (ongoing)  → Interview Loop + Offers     → Jul 2027+
```

Total Duration: ~12 months → Placement Ready

PHASE 0: FOUNDATION & C++ STL (Jun 25 – Jul 15, 2026)

Week 1: Jun 25 – Jul 1 — C++ for CP & STL Containers

Day 1 (Jun 25): Arrays, Vectors, Strings

```
Morning (2h):
  - vector: push_back, pop_back, insert, erase, resize, reserve, shrink_to_fit
  - array (C++11): fixed-size alternative
  - string: substr, find, rfind, erase, replace, append, compare
  - stringstream: parsing mixed input

Evening (2h):
  - Implement: reverse a string in-place, palindrome check
  - Implement: rotate array by K, remove duplicates from sorted array
  - LeetCode: 344, 125, 189, 26
```

Day 2 (Jun 26): Set & Map

```
Morning (2h):
  - set / multiset: insert, erase, find, count, lower_bound, upper_bound
  - map / multimap: insert, [], at, find, erase
  - unordered_set / unordered_map: when to use vs ordered

Evening (2h):
  - Implement: find frequency of elements in array
  - Implement: intersection of two arrays (both sorted & unsorted)
```

```
  - LeetCode: 217, 349, 350, 1 (Two Sum)
```

Day 3 (Jun 27): Stack, Queue, Deque, Priority Queue

```
Morning (2h):
  - stack: push, pop, top, empty
  - queue: push, pop, front, back
  - deque: push_front, push_back, pop_front, pop_back
  - priority_queue: max-heap / min-heap, custom comparator

Evening (2h):
  - Implement: min stack, valid parentheses
  - Implement: sliding window max using deque
  - LeetCode: 20, 155, 239, 703
```

Day 4 (Jun 28): List, Forward List, Pair, Tuple

```
Morning (2h):
  - list: splice, merge, remove, unique, sort
  - forward_list: singly linked list operations
  - pair: make_pair, tie, comparison
  - tuple: make_tuple, get, tie, structured bindings (C++17)

Evening (2h):
  - Practice: all pair/tuple operations
  - LeetCode: 1472 (Design Browser History) - list practice
```

Day 5 (Jun 29): STL Algorithms — Part 1

```
Morning (2h):
  - sort, stable_sort, partial_sort, nth_element
  - reverse, rotate, next_permutation, prev_permutation
  - unique, remove, remove_if
  - binary_search, lower_bound, upper_bound, equal_range

Evening (2h):
  - Implement: manual next_permutation, find kth smallest using nth_element
  - LeetCode: 31 (Next Permutation), 215 (Kth Largest)
```

Day 6 (Jun 30): STL Algorithms — Part 2

```
Morning (2h):
  - accumulate, inner_product, partial_sum, adjacent_difference
  - count, count_if, find, find_if, find_end, search
  - transform, for_each, copy, copy_if
  - merge, inplace_merge, set_intersection, set_union, includes
  - fill, generate, iota, all_of, any_of, none_of

Evening (2h):
```

```
  - Practice: prefix sum using partial_sum
  - Practice: dot product using inner_product
  - LeetCode: 724 (Pivot Index), 238 (Product of Array Except Self)
```

Day 7 (Jul 1): Review + Custom Comparators + Lambdas

```
Morning (2h):
  - Lambda syntax: [capture](params){body}
  - Custom comparators for sort, set, map, priority_queue
  - Sorting vector of pairs, vector of custom objects

Evening (2h):
  - Review all 24 STL functions learned this week
  - Implement: sort by frequency, sort by multiple keys
  - LeetCode: 451, 1366, 1636
```

End of Week 1 Target: LeetCode ~65 solved | STL muscle memory built

---

Week 2: Jul 2–8 — Time/Space Complexity + I/O + Debugging

Day 8-9 (Jul 2-3): Complexity Analysis

```
Topics:
  - Big-O, Theta, Omega formal definitions
  - Amortized analysis: vector push_back, deque operations
  - Master theorem: T(n) = aT(n/b) + f(n)
  - Common complexities: O(1), O(log n), O(n), O(n log n), O(n²), O(2ⁿ), O(n!)
  - Complexity of each STL operation
  - Space-time tradeoffs
  - Recursion tree analysis

Practice: Analyze TC/SC of every code you write - no exceptions
```

Day 10-11 (Jul 4-5): I/O Optimization & Debugging

```
Topics:
  - ios_base::sync_with_stdio(false), cin.tie(NULL), cout.tie(NULL)
  - endl vs '\n', tie with cerr
  - File I/O: freopen for CP
  - Debug macros: #define dbg(x) cerr << #x << " = " << x << endl
  - Debugging checklist: off-by-one, overflow, uninit, bounds
  - Stress testing: brute force vs optimized for small n
  - Print debugging, cerr, conditional debug

Practice: Set up your debug template. Solve 2 problems with stress testing.
```

Day 12-13 (Jul 6-7): Problem Solving Fundamentals

```
Topics:
```

```
  - Reading problem statements correctly (2-3 times)
  - Identifying input/output patterns
  - Brute force → Better → Optimal progression
  - Edge cases: empty, min/max, overflow, negative
  - Multi-test case handling: while(t--)
  - Writing clean CP code

Practice: Solve 5 Codeforces 800-rated problems
```

Day 14 (Jul 8): Phase 0 Review

```
- Re-solve 5 problems from this week from scratch
- Review complexity of all STL operations
- Identify weak areas → note them
```

End of Week 2 Target: LeetCode ~70 | CF ~5 solved | Complexity intuition built

---

Week 3: Jul 9–15 — Recursion + Pointers + Patterns

Day 15-16 (Jul 9-10): Recursion Foundation

```
Topics:
  - Base case + recurrence relation
  - Recursion tree drawing
  - Tail recursion vs non-tail
  - Stack overflow: iterative conversion
  - Parameter passing: by value vs by reference

Practice:
  - Factorial, Fibonacci, Power (fast exponentiation)
  - Print 1 to N, N to 1
  - Sum of array elements, check sorted array
  - Subsequences: print all subsequences
  - LeetCode: 231, 326, 342, 509
```

Day 17-18 (Jul 11-12): Pointers & References Deep Dive

```
Topics:
  - Pointer arithmetic, double pointers, void pointers
  - Reference variables, const reference
  - Function pointers (useful for comparators)
  - Dynamic memory: new/delete vs malloc/free
  - Memory layout: stack, heap, data, text
  - Nullptr vs NULL, dangling pointers, memory leaks

Practice: Write programs demonstrating each concept
```

Day 19-20 (Jul 13-14): Pattern Printing Mastery

```
Topics:
  - Square/Rectangle, Triangle (all orientations)
  - Number patterns, Diamond, Hollow patterns
  - Symmetry patterns, Pascal's triangle
  - Pattern = nested loop intuition = recursion/DP loop intuition

Practice: 22 patterns from Striver A2Z Step 1.3
```

Day 21 (Jul 15): Phase 0 Review + Assessment

```
- Solve 10 mixed problems (random topics from Phase 0)
- Fill weekly review template
- Phase 0 complete
```

End of Phase 0: LeetCode ~80 | CF ~10 | STL fluent | Recursion basics solid

---

PHASE 1: CORE DSA — STRIVER A2Z (Jul 16 – Oct 21, 2026)

Goal: Complete Striver A2Z 474 problems in 14 weeks.
Pacing: 6-7 problems/day, 6 days/week. Sunday = contest + revision.
Parallel: Start Codeforces contests from Week 5 onwards.

Week 4: Jul 16–22 — Step 1 (Learn Basics) + Step 2 (Sorting)

Jul 16-17: Basics (Step 1.1-1.3)

```
- 1.1 C++ basics (6 problems) - quick review
- 1.2 Conditionals (4 problems)
- 1.3 Patterns (22 problems) - revise from Phase 0
```

Jul 18-19: Functions + Arrays (Step 1.4-1.5)

```
- 1.4 Functions (3 problems) - pass by ref vs value
- 1.5 Arrays (5 problems) - 1D/2D, row-major vs column-major
```

Jul 20-21: Strings + Pointers + Recursion (Step 1.6-1.8)

```
- 1.6 Strings (4 problems)
- 1.7 Pointers (6 problems)
- 1.8 Recursion (4 problems)
```

Jul 22: Sorting — Step 2 (7 problems)

```
- Selection, Bubble, Insertion (implement by hand)
- Merge Sort, Quick Sort, Recursive Bubble, Recursive Insertion
- Understand: stable vs unstable, in-place vs out-of-place
```

Target this week: 61 problems. Daily: 8-9 problems.

Week 5: Jul 23–29 — Step 3.1 Arrays Easy (23 problems)

Jul 23-25: Array Easy — Part 1

```
- Largest, Second Largest, Check Sorted, Remove Duplicates
- Rotate array (left, right by K), Move Zeroes
- Linear Search, Union/Intersection
```

Jul 26-28: Array Easy — Part 2

```
- Missing Number, Max Consecutive Ones, Single Number (XOR)
- Longest Subarray with sum K (positive + negatives)
- Two Sum (all variations)
```

Jul 29: Revision + Contest

```
- Re-solve weakest 5 problems from this week
- Participate in LeetCode Weekly Contest (unrated is fine)
```

Target this week: 23 problems.
CF Practice: Start solving 800-rated problems (2/day)

Week 6: Jul 30 – Aug 5 — Step 3.2 Arrays Medium (17 problems)

Jul 30-31: Array Medium — Part 1

```
- Sort Colors (Dutch National Flag) [IMP]
- Majority Element (Moore's Voting: >N/2)
- Kadane's Algorithm (max subarray sum + circular)
- Stock Buy and Sell (1 transaction)
```

Aug 1-3: Array Medium — Part 2

```
- Next Permutation (STL impl + manual)
- Leaders in Array, Longest Consecutive Sequence
- Set Matrix Zeroes (O(1) space)
- Rotate Image (Transpose+Reverse)
```

Aug 4-5: Array Medium — Part 3

```
- Spiral Matrix, Subarray Sum Zero, Subarray XOR
- Majority Element (>N/3)
```

Target this week: 17 problems. Start pattern recognition journal.

Week 7: Aug 6–12 — Step 3.3 Arrays Hard (8 problems)

Aug 6-9: Array Hard

- Pascal's Triangle (row, full, O(n) row)
- 3-Sum, 4-Sum (handle duplicates without set)
- Longest Subarray with Zero Sum
- Subarray XOR equals K
- Merge Overlapping Subintervals
- Merge Two Sorted Arrays (gap method)
- Find Duplicate Number (Floyd's cycle)

Aug 10-12: Deep Revision

- Each problem: learn 2+ approaches
- Re-solve all Array Medium + Hard from scratch
- Participate in LeetCode Weekly Contest

Target this week: 8 problems (each with 2 approaches) + revision of 17 array medium.

Week 8: Aug 13–19 — Step 4 Binary Search — Part 1 (15 problems)

Aug 13-15: BS Basics + 1D

- Standard BS, Lower/Upper Bound
- Search in Rotated Array (I & II)
- Find Min in Rotated Array, Find Peak Element
- Find First and Last Position

Aug 16-18: BS on Answer

- Kth Missing Positive, Aggressive Cows
- Book Allocation, Split Array Largest Sum
- Painters Partition, Koko Eating Bananas
- Nth Root, Minimum Days to Bouquets

Aug 19: Practice + Contest

- Solve 5 random BS problems on Codeforces (1000-1100)
- LeetCode Weekly Contest

Codeforces Rating Target this week: Start solving 900-1000 rated.

Week 9: Aug 20–26 — Step 4 Binary Search — Part 2 (17 problems)

Aug 20-22: BS 2D + Advanced

- Search a 2D Matrix (I & II)
- Find Peak Element in 2D (O(n+m) and O(n log m))

- Median in Row-Wise Sorted Matrix
- Kth Element in Two Sorted Arrays

Aug 23-25: BS on Search Space

- Gas Station, Capacity to Ship Packages
- Minimize Max Distance to Gas Station
- Split Array, Median of Two Sorted Arrays (O(log min(n,m)))
- Allocate Minimum Pages (revisited)

Aug 26: Revision + Contest

- CF Virtual Contest (Div. 2 - solve A + B)
- LeetCode Weekly Contest

A2Z Progress: ~141 problems done (30% of A2Z)

Week 10: Aug 27 – Sep 2 — Step 5 Strings (15 problems)

Aug 27-29: String Basics + Medium

- Reverse Words, Longest Palindromic Substring
- Roman to Integer, Integer to Roman
- ATOI Implementation, Longest Common Prefix
- Anagram check (char count + sort)

Aug 30-Sep 1: String Advanced

- Rabin-Karp (rolling hash), Z-Function, KMP
- Count Substrings with K Distinct Characters
- Minimum Characters to Make Palindrome
- Check if String is Rotation

Sep 2: Practice + Contest

- Re-solve KMP/Z from memory (blank page, no reference)
- LeetCode Weekly Contest

Practice: Rolling hash, prefix function, Z-function by heart.

Week 11: Sep 3–9 — Step 6 Linked List (31 problems)

Sep 3-5: Single LL (9 problems)

- Reverse LL (iterative + recursive), Middle of LL
- Merge Two Sorted LL, Remove Nth from End
- Add Two Numbers (I & II), Intersection
- Palindrome LL, Detect Cycle, Cycle Start Node

Sep 6-7: Double LL (5) + Medium/Hard Start (8)

- DLL: Implementation, Reverse, Delete, Find Pairs
- Flatten LL, Rotate LL, Clone LL with Random
- LRU Cache (MUST DO - appears in every interview)

Sep 8-9: Linked List Hard (9)

- LFU Cache, Merge K Sorted Lists
- Reverse in K-Group, Copy with Random Pointer
- LeetCode Weekly Contest

IMP: LRU Cache = write from scratch, memorize pattern.

Week 12: Sep 10–16 — Step 7 Recursion (25 problems)

Sep 10-12: Subsets + Combinations

- Subsets (I & II, with duplicates)
- Combination Sum (I, II, III)
- Subsequences, All Subsets with bitmask

Sep 13-14: Permutations + Backtracking

- Permutations (I & II)
- N-Queens, N-Queens II
- Sudoku Solver, Rat in a Maze

Sep 15-16: Advanced Recursion

- M-Coloring, Palindrome Partitioning
- Kth Permutation Sequence, Tower of Hanoi
- Generate Parentheses, Expression Add Operators
- LeetCode Weekly Contest

Draw recursion trees for EVERY problem.

Week 13: Sep 17–23 — Step 8 Bit Manipulation (18 problems)

Sep 17-19: Bit Manipulation Basics

- AND, OR, XOR, NOT, shifts
- Check/set/unset/toggle bits, count set bits (Brian Kernighan)
- Power of Two/Four, Single Number (I, II, III)
- XOR of range, Divide without /

Sep 20-22: Bit Manipulation Advanced

```
- All pairs XOR sum, Gray Code
- Subsets using bitmask, Min bit flips
- Prime number of set bits, Swap without temp
```

Sep 23: Practice + Contest

```
- Revise: x & -x (lowest set bit), x & (x-1) (clear lowest set bit)
- LeetCode Weekly Contest
```

Week 14: Sep 24–30 — Step 9 Stack & Queue (30 problems)

Sep 24-26: Stack/Queue Implementation + Conversions

```
- Implement using array, LL, Queue using Stack, Stack using Queue
- Min Stack, Max Stack
- Prefix/Infix/Postfix: all conversions + evaluations
```

Sep 27-29: Monotonic Stack (14 problems)

```
- Next Greater Element (I, II, III)
- Previous Greater/Smaller Element
- Largest Rectangle in Histogram, Maximal Rectangle
- Trapping Rain Water, Sliding Window Maximum
- Remove K Digits, Sum of Subarray Minimums
```

Sep 30: Revision + Contest

- CF Virtual Contest (target: solve A+B)
- LeetCode Weekly Contest

Week 15: Oct 1–7 — Step 10 Sliding Window & Two Pointer (12) + Step 11 Heaps (17)

Oct 1-3: Sliding Window

```
- Fixed: Max Sum Subarray, First Negative in K-size
- Variable: Longest Substring Without Repeating, Fruits into Basket
- Longest K Unique, Min Window Substring (hard)
- Count Subarrays with given XOR/Sum
```

Oct 4-6: Heaps

```
- Kth Largest/Smallest, K Largest/Smallest Elements
- Sort K Sorted, Merge K Sorted, K Closest Points
- Top K Frequent, Connect Ropes (Huffman)
- Median in Stream, Sliding Window Median
```

- Task Scheduler, Rearrange String K Apart

Oct 7: Practice + Contest

Week 16: Oct 8–14 — Step 12 Greedy (15) + Step 13 Trees Start

Oct 8-10: Greedy

- N Meetings, Activity Selection, Min Coins
- Fractional KnapSack, Job Sequencing
- Min Platforms, Jump Game (I & II)
- Candy Distribution, Water the Garden
- Non-Overlapping Intervals, Insert Interval, Min Arrows

Oct 11-13: Tree Traversals (12 problems)

- Inorder, Preorder, Postorder (recursive + iterative)
- Level Order, Spiral/Zigzag, Boundary, Vertical
- Top/Bottom/Left/Right View, Diagonal
- Morris Traversal (O(1) space) - IMP

Oct 14: Contest + Tree practice

Week 17: Oct 15–21 — Step 13 Trees (contd) + Step 14 BST

Oct 15-17: Tree Medium (12 problems)

- Height, Diameter, Balanced, Same Tree
- Subtree, Symmetric, Max Path Sum
- LCA in BT, Children Sum Property
- Count Nodes in Complete BT

Oct 18-19: Tree Hard (14 problems)

- Serialize/Deserialize BT, All Nodes at Distance K
- Kth Ancestor, Construct from Inorder+Preorder/Postorder
- Flatten BT to LL, Burning Tree, Max Width

Oct 20-21: BST (16 problems)

- Search, Insert, Delete, Floor/Ceil
- Validate BST, LCA in BST, Kth Smallest/Largest
- Two Sum in BST, BST Iterator
- Recover BST, Serialize/Deserialize BST

End of Phase 1: ~374 problems done (79% of A2Z) | CF: ~800 rating

---

PHASE 2: CS FUNDAMENTALS + LLD (Oct 22 – Dec 16, 2026)

Goal: Master CS fundamentals (OS, DBMS, SQL, CN, OOP) + Low-Level Design for interviews.

Week 18: Oct 22–28 — Step 15 Tries + Step 16 Graphs Start

Oct 22-23: Tries (7 problems)

- Trie Implementation (insert, search, startsWith)
- Longest Common Prefix, Longest Word with All Prefixes
- Count Distinct Substrings, Maximum XOR of Two Numbers
- Word Break, Word Search II

Oct 24-28: Graph Learning (9 problems)

- Adj List/Matrix, BFS, DFS (connected/disconnected)
- Connected Components, Cycle Detection (BFS+DFS)
- Bipartite Graph Check

Week 19: Oct 29 – Nov 4 — Graphs Medium (15 problems)

- Topological Sort (DFS + Kahn's Algorithm)
- Shortest Path in DAG, Dijkstra's Algorithm
- Prim's MST, Kruskal's MST (DSU)
- Bridges in Graph (Tarjan's), Articulation Points
- Kosaraju's (SCC), Bellman-Ford, Floyd-Warshall

Week 20: Nov 5–11 — Graphs Hard (18 problems)

- Word Ladder (I & II), Alien Dictionary
- Clone Graph, Course Schedule (I, II, IV)
- Cheapest Flights, Swim in Rising Water
- Path with Min Effort, Number of Islands (I, II)
- Max Area of Island, Rotting Oranges
- Surrounded Regions, Pacific Atlantic Water Flow
- Accounts Merge, Evaluate Division

Week 21: Nov 12–18 — DP — Part 1 (1D + 2D/3D)

Nov 12-14: 1D DP (12 problems)

- Fibonacci (memo → tab → space opt)
- Climbing Stairs, Frog Jump (K jumps)
- House Robber (I, II, III), Ninja's Training

- Unique Paths (I & II), Min Path Sum
- Triangle, Max Sum Increasing Subsequence

Nov 15-17: 2D/3D DP (8 problems)

- Cherry Pickup (I & II), Chocolate Pickup
- Min/Max Falling Path Sum, Dungeon Game

Nov 18: Practice + Contest

Week 22: Nov 19–25 — DP — Part 2 (Subsequences + Strings)

Nov 19-21: DP on Subsequences (10 problems)

- Subset Sum Equal to K, Partition Equal Subset Sum
- Count Subsets with Sum K, Min Subset Sum Diff
- Target Sum, Coin Change (I & II)
- 0/1 Knapsack, Unbounded Knapsack
- Rod Cutting, Buy & Sell Stock (all variants)

Nov 22-24: DP on Strings (8 problems)

- LCS, Print LCS, Shortest Common Supersequence
- Longest Palindromic Subsequence, Min Insertions for Palindrome
- Edit Distance, Wildcard Matching, Distinct Subsequences

Nov 25: Practice + Contest

Week 23: Nov 26 – Dec 2 — DP — Part 3 (LIS + Partition + Squares + Stocks)

Nov 26-27: DP on LIS (5)

- LIS ($O(n^2)$ + $O(n \log n)$ binary search)
- Printing LIS, Number of LIS
- LIS on Strings, Longest Bitonic Sequence

Nov 28-29: DP on Partition (6) + Squares (3)

- MCM, Print MCM, Burst Balloons
- Egg Dropping, Boolean Parenthesization
- Min Cost to Cut Stick
- Max Rectangle with 1s, Count Square Submatrices, Max Square

Nov 30-Dec 1: DP on Stocks (3)

- All 6 stock problems from 1 template

Dec 2: DP Revision + Contest

A2Z Complete | LeetCode: ~450 | CF: ~900-1000

Week 24: Dec 3–9 — OPERATING SYSTEMS

Day 1-2: Process & Threads

```
Topics:
  - Process lifecycle (new, ready, running, wait, terminated)
  - Process vs Thread (memory, overhead, communication)
  - Context switch, PCB, System calls
  - Thread models: user-level vs kernel-level
  - Multithreading models: many-to-one, one-to-one, many-to-many

Practice:
  - Write a C++ program with fork() (Unix)
  - Write a multi-threaded program with mutex
```

Day 3-4: CPU Scheduling + Synchronization

```
Topics:
  - Scheduling: FCFS, SJF, Round Robin, Priority, Multilevel Queue
  - Convoy effect, Starvation, Aging
  - Race condition, Critical section
  - Peterson's solution, Test-and-Set, Mutex, Semaphore
  - Producer-Consumer, Reader-Writer, Dining Philosopher

Practice:
  - Implement producer-consumer with condition_variable
  - Solve synchronization problems
```

Day 5-6: Deadlock + Memory Management

```
Topics:
  - Deadlock: 4 conditions, Prevention, Avoidance (Banker's)
  - Detection, Recovery
  - Memory: Contiguous allocation, Fragmentation
  - Paging, Segmentation, Virtual Memory
  - Page replacement: FIFO, Optimal, LRU, Clock
  - Thrashing, Working set, TLB

Common Interview Questions:
  - What happens when you type a URL? (full flow)
  - Difference: paging vs segmentation
  - How does virtual memory work?
  - Starvation prevention?
  - Deadlock conditions + prevention
```

Day 7: OS Revision

```
- Re-solve: producer-consumer from scratch
- Re-solve: Banker's algorithm manually
- Mock: Explain virtual memory to a 5-year-old
```

Week 25: Dec 10–16 — DBMS + SQL

Day 1-2: DBMS Concepts

```
Topics:
  - ACID properties, Transactions
  - Normalization: 1NF, 2NF, 3NF, BCNF (with examples)
  - Indexing: B-tree, Hash index, Clustered vs Non-clustered
  - Joins: INNER, LEFT, RIGHT, FULL, CROSS, SELF
  - Keys: Primary, Foreign, Candidate, Super, Composite
  - ER Diagrams: entities, relationships, cardinality

Practice:
  - Draw ER diagram for Library Management System
  - Normalize a denormalized table to 3NF
```

Day 3-4: SQL Practice — Part 1

```
-- Must-know queries:
SELECT, WHERE, ORDER BY, GROUP BY, HAVING, LIMIT/OFFSET
-- Joins (all types), Subqueries (correlated + non-correlated)
-- Aggregations: COUNT, SUM, AVG, MIN, MAX
-- String functions: CONCAT, SUBSTRING, REPLACE, UPPER, LOWER
-- Date functions: DATE_DIFF, DATE_ADD, EXTRACT

Practice Problems:
  1. Second highest salary (OFFSET or subquery)
  2. Department-wise average salary > 50k
  3. Employees earning more than their managers (SELF JOIN)
  4. Duplicate emails
  5. Customers who ordered in last 30 days but not before
```

Day 5-6: SQL Practice — Part 2 (Window Functions + CTEs)

```
-- WINDOW FUNCTIONS:
ROW_NUMBER(), RANK(), DENSE_RANK()
LEAD(), LAG(), FIRST_VALUE(), LAST_VALUE()
SUM() OVER(PARTITION BY ... ORDER BY ...)

-- CTEs (Common Table Expressions):
WITH cte AS (...)
```

```
-- RECURSIVE CTEs:
WITH RECURSIVE cte AS (...)

Practice Problems:
  1. Nth highest salary (DENSE_RANK)
  2. Running total by department
  3. Find gaps in sequence
  4. Organizational hierarchy (recursive CTE)
  5. Top 3 products by category
```

Day 7: SQL + DBMS Revision

```
- Solve 10 SQL problems on LeetCode (Database section)
- Mock: Explain ACID, Normalization, Indexing
- Re-solve: 5 most important SQL queries from memory
```

Week 26: Dec 17–23 — COMPUTER NETWORKS + OOP

Day 1-2: CN Basics

```
Topics:
  - OSI model (7 layers) - what happens at each layer
  - TCP/IP model (4 layers)
  - TCP vs UDP (headers, 3-way handshake, flow control, congestion control)
  - HTTP/1.1 vs HTTP/2 vs HTTP/3
  - DNS resolution (full flow)
  - IP addressing, Subnetting (CIDR)
  - HTTP methods (GET, POST, PUT, PATCH, DELETE)
  - HTTP status codes (1xx, 2xx, 3xx, 4xx, 5xx)
  - Cookies vs Sessions, JWT basics

Common Questions:
  - What happens when you type google.com? (ULTIMATE question)
  - TCP 3-way handshake
  - Difference: TCP vs UDP
  - HTTP vs HTTPS (SSL/TLS handshake)
  - DNS resolution steps
```

Day 3-4: OOP Concepts + C++ OOP Deep Dive

```
Topics:
  - Encapsulation, Abstraction, Inheritance, Polymorphism
  - Virtual functions, vtable, vptr (C++ specific)
  - Abstract classes vs Interfaces (pure virtual)
  - Multiple inheritance + Diamond problem (virtual inheritance)
  - static, const, mutable, explicit keywords
  - Friend functions/classes, Operator overloading
  - Copy constructor, Copy assignment, Move constructor, Move assignment
```

```
  - Rule of 3 / Rule of 5
  - Virtual destructors (WHY? - demonstrate with example)
  - Slicing problem + how to prevent

Practice:
  - Implement: Shape hierarchy (Circle, Rectangle, Triangle)
  - Implement: RAII wrapper for dynamically allocated array
  - Implement: Thread-safe Singleton
  - Write vtable layout for a class hierarchy on paper
```

Day 5-6: OOP Design Principles

```
SOLID:
  S - Single Responsibility: Invoice → InvoiceCalculator + InvoicePrinter
  O - Open/Closed: Add new shapes without modifying AreaCalculator
  L - Liskov: Rectangle vs Square (violation and fix)
  I - Interface Segregation: Split fat interfaces
  D - Dependency Inversion: Depend on abstractions

Design Patterns (implement each in C++):
  - Singleton (thread-safe with double-checked locking)
  - Factory Method
  - Strategy (payment methods)
  - Observer (event system)
  - Decorator (pizza toppings)
  - Adapter (legacy integration)
```

Day 7: CN + OOP Revision

```
  - Write all 7 OSI layers + protocols at each from memory
  - Implement: Strategy pattern for sorting algorithms
  - Mock: "Explain OOP pillars" + "What happens when you type a URL"
```

Week 27: Dec 24–30 — LOW-LEVEL DESIGN PRACTICE

Dec 24-25: LLD — Easy Problems

```
1. Design a Parking Lot
   - Multiple floors, vehicle types, spots
   - ParkingStrategy (nearest, cheapest)
   - Ticket generation, fee calculation

2. Design a Vending Machine
   - States: idle, select, dispense, refund
   - Inventory management, payment handling
   - State pattern implementation

3. Design a Library Management System
```

```
   - Book, Member, Librarian entities
   - Search by title/author, checkout/return
   - Fine calculation, reservation system
```

Dec 26-28: LLD — Medium Problems

```
4. Design a Chess Game / Tic-Tac-Toe
   - Board, Piece hierarchy (King, Queen, etc.)
   - Move validation, Check/Checkmate detection
   - Player turns, game state management

5. Design an ATM
   - Card reading, PIN validation
   - Account types (Checking, Savings)
   - Withdraw, Deposit, Balance, Transfer
   - Cash dispensing logic

6. Design Splitwise / Expense Sharing
   - User, Expense, Group entities
   - Split types: equal, exact, percent
   - Balance simplification (min transactions)
   - Settle up logic
```

Dec 29-30: LLD — Hard Problems

```
7. Design Movie Ticket Booking
   - Movie, Theater, Screen, Seat entities
   - Show timing, seat selection (hold → booked)
   - Payment processing, Booking confirmation
   - Concurrent booking handling

8. Design a Cache (LRU + LFU + TTL)
   - Eviction policy interface
   - LRU: Doubly Linked List + HashMap
   - LFU: Frequency buckets
   - Thread safety considerations

9. Design a Pub-Sub System
   - Topic, Publisher, Subscriber, Message
   - Push vs Pull models
   - Async message delivery
   - Ack mechanism for reliability
```

End of Phase 2: LeetCode: ~500 | A2Z complete | OS/DBMS/CN/OOP solid | LLD: 9 designs

---

PHASE 3: SYSTEM DESIGN + ADVANCED (Dec 17 – Feb 10, 2027)

Week 28: Dec 31 – Jan 6 — System Design Fundamentals

Day 1-2: Design Basics

```
Topics:
  - CAP Theorem (Consistency, Availability, Partition Tolerance)
  - Consistency models: Strong, Eventual, Causal
  - Load Balancers: Layer 4 vs Layer 7, Algorithms (RR, Least Connections, IP Hash)
  - Caching: Redis/Memcached, Cache strategies (Read-through, Write-through, Write-behind)
  - Cache eviction: LRU, LFU, TTL
  - CDN: How it works, Edge servers, Origin pull

Practice:
  - Draw block diagram of a CDN
  - Explain CAP theorem with real examples
  - Compare: Read-through vs Write-through vs Write-behind caching
```

Day 3-4: Databases + Storage

```
Topics:
  - SQL vs NoSQL (when to use which)
  - Sharding: Horizontal partitioning, Consistent Hashing
  - Replication: Leader-Follower, Multi-leader, Quorum
  - Indexing: B-tree, LSM-tree (how they differ)
  - Data warehouses vs Data lakes
  - Blob storage (S3): how it works

Practice:
  - Design a sharding strategy for a social media DB
  - Explain consistent hashing with ring diagram
  - When would you pick NoSQL over SQL?
```

Day 5-6: Communication + Message Queues

```
Topics:
  - REST vs gRPC (when to use each)
  - WebSockets: real-time bidirectional
  - Message Queues: Kafka vs RabbitMQ
  - Pub-Sub pattern
  - Polling vs Long-polling vs WebSockets vs SSE
  - RPC vs REST vs GraphQL

Practice:
  - Design a rate limiter (token bucket, sliding window)
```

```
  - When would you use Kafka vs RabbitMQ?
  - Design a chat system (WebSockets vs polling)
```

Day 7: System Design Mini-Designs

```
  - Design a URL Shortener (all components, end-to-end)
  - Practice: Draw the full architecture on paper in 30 min
```

Week 29: Jan 7–13 — System Design Deep Dive — Part 1

Jan 7-8: Design URL Shortener (TinyURL)

```
Requirements:
  - Shorten long URLs, redirect to original
  - Traffic: 100M URLs/month, 1B redirects/month
  - 1% read:write ratio

Deep Dive:
  - Encoding: Base62 vs MD5 hash
  - Unique ID generation: Snowflake, DB sequence, UUID
  - Database: SQL (URL → short code), shard by hash
  - Caching: Redis for hot URLs (LRU)
  - Redirection: 301 vs 302 (permanent vs temporary)
  - Rate limiting on write path

Key Tradeoffs:
  - Hash collision handling
  - Custom short URLs
  - Analytics tracking
```

Jan 9-10: Design WhatsApp / Chat System

```
Requirements:
  - 1-on-1 + group chat, 1B+ users
  - Messages: text, images, videos
  - Last seen, read receipts, online/offline
  - End-to-end encryption

Deep Dive:
  - WebSocket connections (persistent)
  - Message storage: Inbox (per user), Message table
  - Push notification: APNS/FCM fallback
  - Group chat: fan-out on write (small groups) vs pull (large groups)
  - Media: CDN + CDN for images/videos
  - Encryption: Signal protocol (X3DH + Double Ratchet)

Key Tradeoffs:
  - Fan-out on write vs pull for groups (threshold: 100 members)
```

```
  - Message ordering (server timestamp vs client timestamp)
```

Jan 11-12: Design Netflix / YouTube

```
Requirements:
  - 200M+ users, billions of hours watched/day
  - Upload, transcode, stream video
  - Adaptive bitrate, recommendations
  - Global low latency

Deep Dive:
  - Video ingestion: upload → encode (multiple resolutions) → CDN
  - Adaptive bitrate (ABR): HLS/DASH protocol
  - CDN: Edge servers cache popular content
  - Recommendation: collaborative filtering + content-based
  - Pre-processing: thumbnail generation, content moderation

Key Tradeoffs:
  - Store all resolutions vs transcode on-the-fly
  - Cache popular vs pre-fetch based on user behavior
  - Upload latency vs quality
```

Jan 13: Revision + Design from scratch

```
  - Design URL Shortener from memory (30 min)
  - Design Chat System from memory (45 min)
```

Week 30: Jan 14–20 — System Design Deep Dive — Part 2

Jan 14-15: Design Uber

```
Requirements:
  - Real-time driver-passenger matching
  - ETA estimation, surge pricing
  - Location tracking, trip history
  - 50M+ users, 10M+ drivers

Deep Dive:
  - Geospatial indexing: QuadTree / Geohash / Google S2
  - Matching: nearest driver (within radius) → dispatch
  - ETA: Google Maps API + historical data
  - Surge pricing: supply-demand ratio in geohash cells
  - Real-time: WebSocket for driver location pings

Key Tradeoffs:
  - Polling vs push for location updates
  - Accuracy vs speed of nearest driver search
  - Fairness in matching algorithm
```

Jan 16-17: Design Twitter / Newsfeed

```
Requirements:
  - Post tweets, follow users, timeline
  - 500M+ users, 500M tweets/day
  - Newsfeed: recent tweets from followed users
  - Trending topics, search

Deep Dive:
  - Tweet storage: SQL (tweets, users, follows)
  - Timeline generation: fan-out on write (celebrities → pull)
  - Newsfeed: pre-compute vs real-time
  - Trending: sliding window (last hour) → count hashtags
  - Search: Elasticsearch / inverted index

Key Tradeoffs:
  - Fan-out on write (fast reads, slow writes for celebs)
  - Hybrid: follow count threshold (10k = pull, under = push)
```

Jan 18-19: Design Instagram / Dropbox

```
Instagram:
  - Photo/video upload, feed, stories, likes, comments
  - CDN for media, social graph
  - Newsfeed: similar to Twitter + media optimization

Dropbox/Google Drive:
  - File upload/download, sync across devices
  - File chunking (4MB blocks), deduplication (hash)
  - Delta sync: sync only changed parts
  - Conflict resolution: last-writer-wins, version history
  - Storage: S3/Blob + metadata DB

Key Tradeoffs:
  - Client-side vs server-side dedup
  - Delta sync granularity
  - Consistency: strong for metadata, eventual for content
```

Jan 20: Design Practice

```
  - Design Twitter feed from memory (45 min)
  - Design Dropbox from memory (45 min)
```

Week 31: Jan 21–27 — System Design — Remaining

Jan 21-22: Design Web Crawler + Rate Limiter

```
Web Crawler:
```

```
  - BFS frontier, URL frontier priority (PageRank)
  - URL dedup: Bloom filter (false positives OK) + set
  - Politeness: delay between requests per domain
  - Robustness: handle crashes (checkpoint), malformed HTML
  - Storage: raw HTML, extracted text, metadata

Rate Limiter:
  - Token bucket: refill rate + bucket size
  - Sliding window: atomic counters per window
  - Distributed: Redis sorted sets with TTL
  - Rules: per user, per API, per endpoint
  - Response: 429 Too Many Requests + Retry-After header

Key Tradeoffs:
  - Accuracy vs memory (sliding window vs token bucket)
  - Distributed consistency (Redis cluster)
  - Handling burst traffic
```

Jan 23-24: Design Distributed Cache + Search Engine

```
Distributed Cache:
  - Consistent hashing (minimize rehashing on add/remove)
  - Replication: each key on N nodes
  - Failure handling: gossip protocol
  - Cache-aside, Read-through, Write-through strategies
  - Redis Cluster architecture

Search Engine (mini-Google):
  - Crawl → Index (inverted index) → Rank (PageRank) → Serve
  - Inverted index: term → list of document IDs
  - Ranking: TF-IDF, PageRank, recency, personalization
  - Query processing: tokenization, stemming, query expansion
  - Serving: index sharding, replication for read throughput
```

Jan 25-27: System Design Revision

```
  - Re-design: Uber, Twitter, Netflix from memory (each in 30 min)
  - Practice: The "10-step framework" (see PLACEMENTS_BEYOND.md)
  - Watch: Gaurav Sen system design videos on remaining topics
```

Week 32: Jan 28 – Feb 3 — DSA Revision — Hard Problems

```
Re-solve the hardest problems from each topic (1-2 per topic):
  - Arrays: 4-Sum, Merge Intervals, Subarray XOR
  - BS: Median of Two Sorted Arrays, Aggressive Cows
  - Strings: KMP, Rabin-Karp, Manacher
  - LL: LFU Cache, Reverse in K-Group
```

```
  - Trees: Max Path Sum, Serialize BT, Burning Tree
  - Graphs: Word Ladder II, Alien Dictionary, Accounts Merge
  - DP: Burst Balloons, Edit Distance, Cherry Pickup
  - Greedy: Candy Distribution, Job Sequencing

Focus: Explanation + Clean Code + Edge Cases
```

Week 33: Feb 4–10 — Company-Specific Problem Patterns

```
Amazon (20 problems):
  - Two Sum variants, LRU Cache, Rotting Oranges
  - Longest Substring Without Repeating
  - Word Ladder, Number of Islands
  - Amazon OA problems (LeetCode tagged)

Google (20 problems):
  - Median of Two Sorted, Text Justification
  - Serialize/Deserialize, Word Search II
  - Cherry Pickup, Dungeon Game
  - Time-based Key-Value Store

Microsoft (15 problems):
  - Reverse K-Group, Alien Dictionary
  - LFU Cache, Interval problems
  - Excel Sheet Column logic

Uber/Stripe/Atlassian (15 problems):
  - Merge Intervals, Design problems
  - Rate Limiter, LRU Cache
  - API design, SQL problems
```

PHASE 4: PROJECTS + RESUME + OPEN SOURCE (Feb 11 – Mar 24, 2027)

Week 34: Feb 11–17 — Project 1: CP-Templates Library

```
Build a production-quality C++ template library:

Structure:
  CP-Templates/
       include/
           data-structures/   (segment-tree.hpp, fenwick.hpp, dsu.hpp)
           algorithms/        (graph.hpp, string.hpp, number-theory.hpp)
```

```
    utils/              (modint.hpp, debug.hpp, fast-io.hpp)
  tests/
    test-all.cpp
  CMakeLists.txt
  README.md

README:
  - Badges: CI passing, C++17, license
  - Quick start: include + example
  - List of all headers with brief description
  - Time/Space complexity for each
  - Link to your LeetCode/CF profiles

Push to GitHub: 10+ stars target
Post on LinkedIn: "I built a CP library"
```

Week 35: Feb 18–24 — Project 2: Full-Stack App (LeetCode Stats Tracker)

```
Tech: React/Node.js (or C++ backend with crow/restinio + React)

Features:
  - Enter LeetCode username → show stats (solved, rating, contest history)
  - Topic-wise breakdown: arrays, trees, DP → progress bars
  - Daily problem recommendation based on weak areas
  - Streak tracker (calendar heatmap)
  - LeetCode API (scraping or graphql)

Why this project:
  - Uses API integration, frontend, backend
  - Relevant to CP journey → shows passion
  - Can expand later (add CF, CodeChef support)
```

Week 36: Feb 25 – Mar 3 — Resume + LinkedIn + GitHub Branding

Resume Building

```
Structure (1 page only):
  Header: Name, Phone, Email, LinkedIn, GitHub, LeetCode/CF
  Education: College, CGPA, Expected graduation
  Skills: Languages (C++, Python, SQL), Tools (Git, Linux), Frameworks
  Projects:
    1. CP-Templates Library (link, description, technologies)
    2. LeetCode Stats Tracker (link, description)
    3. Any other project
  Achievements:
    - LeetCode: X problems solved, Y contest rating
```

```
    - Codeforces: Z rating
    - ICPC/college contests ranked
  Experience (if any): Internship, Teaching Assistant, Open Source
  Extras: Open Source contributions, blog, technical writing

Rules:
  - 1 page MAX (even for experienced)
  - No colors, no photos (ATS-friendly)
  - Quantify everything: "Solved 500+ problems" not "Solved many problems"
  - PDF format only
```

LinkedIn Optimization

```
Headline: "C++ | DSA | Software Engineer | Placement 202X"
About section: Your story in 3 paragraphs (starting point → journey → goal)
Featured: Pin GitHub projects + LeetCode profile
Experience: Projects with description
Skills: C++, DSA, SQL, OOP, System Design, etc.
Recommendations: Ask friends/professors
Activity: Start posting 2-3 times/week

Post schedule:
  - Mon: Problem solution breakdown
  - Wed: Concept explanation (thread)
  - Fri: Weekly progress + contest update
```

GitHub Profile

```
README.md with:
  - Banner: "C++ | DSA | CP" with shields.io badges
  - LeetCode/CF live stats widgets
  - "Currently preparing for placements"
  - Pinned: CP-Templates, Stats Tracker, top repos
  - Contribution graph: green = every day
```

Week 37: Mar 4–10 — Open Source Contribution

```
TARGET: 5 merged PRs in 2 weeks

Week 37:
  - Find 3 beginner-friendly issues on CP-Algorithms or TheAlgorithms/C-Plus-Plus
  - Fix documentation first (easy, teaches workflow)
  - Then fix actual code issues

Process:
  - Fork → Clone → Branch → Code → Test → PR
  - Follow project's CONTRIBUTING.md
```

```
  - Write good PR description: problem, solution, testing
  - Respond to code review feedback
  - After merge, add to resume under "Open Source Contributions"
```

Week 38: Mar 11–17 — Open Source (contd) + Mock Interviews Start

```
  - Continue open source: 2 more PRs this week
  - Start mocks: 1 mock interview/week (friend or Pramp)
  - Practice: Explain DSA solutions aloud (record yourself)
  - Practice: Write code in Google Docs (no auto-complete)
```

Week 39: Mar 18–24 — Behavioral Preparation

```
STAR stories for 10 experiences:
  1. Technical project you led
  2. Challenging bug you fixed
  3. Conflict with teammate
  4. Time you went above and beyond
  5. Failure and what you learned
  6. Decision with incomplete information
  7. Time you disagreed with authority
  8. Most challenging problem you solved
  9. Time you helped someone grow
  10. Leadership experience

For each story (STAR):
  S: Situation - 1-2 sentences context
  T: Task - what was your responsibility
  A: Action - SPECIFIC steps you took (70% of answer)
  R: Result - quantifiable impact

Additionally:
  - "Why do you want to work here?" (company research)
  - "Where do you see yourself in 5 years?"
  - "What's your biggest weakness?" (real + fix)
  - "Tell me about yourself" (60-second pitch)
```

PHASE 5: PLACEMENT PREP + MOCKS (Mar 25 – Jun 30, 2027)

Week 40: Mar 25–31 — Application Sprint + OA Prep

Application Strategy

```
Apply to MINIMUM 30 companies this week:
  - Amazon SDE-1
  - Google SWE
  - Microsoft SWE
  - Uber SWE
  - Stripe SWE
  - Atlassian SWE
  - Goldman Sachs
  - JP Morgan
  - All Indian PBCs (Razorpay, Zeta, Cred, etc.)
  - All startups (Rippling, Notion, etc.)

Track in a spreadsheet:
  | Company | Role | Applied Date | OA Date | Round 1 | Round 2 | Round 3 | Offer |
  |---------|------|--------------|---------|---------|---------|---------|-------|
```

OA Preparation

```
- Solve 2 LeetCode contests/week (timed)
- Focus on speed: Q1 (5 min), Q2 (15 min), Q3 (25 min)
- Company-specific: Solve last 6 months tagged problems
- SQL: 1 hour of SQL practice daily
- Aptitude: 30 min daily on IndiaBix (LR + Quant)
```

Week 41: Apr 1–7 — Mock Interview Bootcamp

```
Daily schedule:
  Morning (3h): DSA mock (1 problem + detailed discussion)
  Afternoon (2h): LLD mock (1 design + coding)
  Evening (2h): System Design mock (1 design)
  Night (1h): Behavioral mock (3 STAR stories)

Platforms:
  - Pramp.com (free peer-to-peer)
  - InterviewBit mock interviews
  - Friend/senior (2-3 rounds/week)
  - Self-record: phone recording your explanation

Feedback template after each mock:
  [ ] Communication (1-5)
  [ ] Problem-solving approach (1-5)
  [ ] Code correctness + cleanliness (1-5)
  [ ] Edge cases covered (1-5)
  [ ] Time management (1-5)
```

```
  [ ] Key improvements needed:
```

Week 42: Apr 8–14 — DSA Rapid Fire + Weak Areas

```
Identify weak areas from mocks:
  - Slow on graph problems? → Solve 5 graph problems/day
  - DP recurrence not clicking? → Re-do DP on subsequences
  - System design unclear? → Redraw 5 architectures

Priority: Focus 70% time on weak areas, 30% on revision of strong areas

Daily target: 5 DSA problems (timed: 30 min/problem)
```

Week 43: Apr 15–21 — LLD + System Design Revision

```
Redo all LLD problems from scratch (timed):
  - Parking Lot (40 min), Vending Machine (40 min)
  - Splitwise (45 min), Pub-Sub (45 min)
  - Cache (40 min), Movie Booking (50 min)

Redo all System Designs from scratch (timed):
  - URL Shortener (45 min), Chat System (45 min)
  - Netflix (45 min), Uber (45 min)
  - Twitter (45 min), Web Crawler (45 min)
```

Week 44: Apr 22–28 — CS Fundamentals Revision

```
OS (1 day):
  - Process/Thread, Scheduling, Deadlock, Paging, Virtual Memory
  - Re-solve: producer-consumer, Banker's algorithm

DBMS + SQL (1 day):
  - ACID, Normalization (1NF→BCNF), Indexing
  - Re-solve: 10 SQL queries (LeetCode Database hard)

CN (1 day):
  - OSI, TCP vs UDP, DNS, HTTP, Load Balancers
  - Re-explain: "What happens when you type a URL"

OOP (1 day):
  - SOLID, Design Patterns (Singleton, Factory, Strategy, Observer)
  - Re-implement: 3 design patterns from scratch

Aptitude + Reasoning (3 days):
```

```
  - P&C, Probability, Time-Speed-Distance, Averages
  - Blood Relations, Seating Arrangement, Syllogisms
  - 2 mock aptitude tests (30 questions in 30 min)
```

Week 45: Apr 29 – May 5 — Full Mock Interview Loops

```
Full loop  2.5 hours (simulate real):
  - Round 1: DSA (45 min) - 1 medium + 1 hard
  - Round 2: DSA/LLD (45 min) - 1 LLD or 1 hard DSA
  - Round 3: System Design / CS Fundamentals (30 min)
  - Behavioral (15 min)

Do 2 full loops this week (one with friend, one on Pramp)

Target:
  - DSA: Solve 2 problems in 40 min with clean code + edge cases
  - LLD: Design + code key classes in 40 min
  - System Design: Clear architecture + tradeoffs in 30 min
  - Behavioral: STAR answers in 90 sec each
```

Week 46: May 6–12 — Company-Specific OA Practice

```
For each company you applied to:
  - Find the exact OA platform (HackerRank, CodeSignal, etc.)
  - Practice on that platform's environment
  - Solve last year's OA questions (LeetCode discuss / GFG)

Focus areas by company:
  - Amazon: LP-heavy + DSA medium-hard
  - Google: Hard DSA + algorithmic thinking
  - Microsoft: DSA + LLD + Go/C# sometimes
  - Uber: DSA + System Design
  - Stripe: DSA + API design + debugging
```

Week 47-48: May 13–26 — OA Season (Peak)

```
Most companies send OAs during this period.

Schedule:
  Morning: Give OAs (expect 4-6 per week)
  Afternoon: Weak area drilling based on OA performance
  Evening: Mock interviews (keep 2-3/week)
  Night: Company-specific preparation for upcoming interviews
```

```
After each OA:
  - Note the questions (memory)
  - Solve them again (concept reinforcement)
  - Identify patterns: what topics keep appearing?
```

Week 49: May 27 – Jun 2 — Interview Preparation (Shortlisting)

```
Some candidates start getting shortlisted for interviews.

Focus:
  - If shortlisted: Company-specific deep dive (read last 20 interview experiences)
  - If not: Don't panic - many companies send invites late
  - Keep applying: Target 10 more companies/week
  - Keep mocks: 3 per week minimum
```

Week 50-51: Jun 3–16 — Interview Season Begins

```
Interviews start rolling in. Your schedule:
  - Day before interview:
    - Light revision of problem patterns
    - Review STAR stories
    - Sleep 8 hours (non-negotiable)

  - Interview day:
    - Light meal 2h before
    - Warm up: 2 easy LeetCode problems
    - No new topics (you know what you know)
    - Confidence: "I've prepared for 12 months"

  - After interview:
    - Note down questions asked
    - Send thank-you email to interviewer
    - Analyze: what could have gone better?
    - Don't dwell - prepare for next
```

Week 52: Jun 17–24 — Offer Season

```
Some offers will start coming. Your actions:
  - Got offer: Negotiate (research levels.fyi, Glassdoor)
  - Didn't get: It's a numbers game. 10 rejections = 1 offer eventually
  - In pipeline: Follow up after 7 days politely
  - No response: Move on, keep interviewing

Negotiation tips:
```

```
  - "I have an offer from X, can you match/beat this?"
  - Never accept first offer always negotiate
  - Consider: Base + Stocks + Bonus + Relocation
  - Don't burn bridges - tech world is small
```

Week 53: Jun 25–30 — Placement Ready

```
If placed:
  - Accept offer
  - Start preparing for joining (reading company tech stack)
  - Help juniors (teaching = reinforcing)
  - Update LinkedIn + GitHub

If not yet placed:
  - Don't give up - placements continue for months
  - Review interview feedback, fix weaknesses
  - Keep applying to newer openings
  - Consider off-campus drives
  - You're prepared. It's a matter of time.
  - The skills you built are permanent.
```

PHASE 6: BEYOND PLACEMENT (Jul 2027+)

```
After placement secured, you have 6-12 months before joining.

Recommended focus:
  1. Deep dive into a domain (distributed systems, ML, databases)
  2. Build a real project (something users will actually use)
  3. Contribute regularly to open source (builds network)
  4. Read: "Designing Data-Intensive Applications" (Kleppmann)
  5. Read: "Clean Code" + "Clean Architecture" (Uncle Bob)
  6. LeetCode: Continue solving (keep muscle memory)
  7. Learn your joining company's tech stack (Go, Java, Python, etc.)
  8. Internship → Pre-placement offer if you perform well
```

WEEKLY REVIEW TEMPLATE

```
WEEK STARTING: ________

PROBLEMS SOLVED: ____
  - LeetCode: ____
  - Codeforces: ____
  - Other: ____
```

```
TOPICS COVERED:
  -

WEAK AREAS IDENTIFIED:
  -

MISTAKES REPEATED:
  -

CONTEST PERFORMANCE:
  - Rank: ____ | Rating Change: ____
  - What went well: ____
  - What to improve: ____

PLAN FOR NEXT WEEK:
  -

NOTES / CONFIDENCE LEVEL (1-10): ____
```

KEY METRICS TO TRACK

```
LeetCode:
  - Total solved: ____ (Target: 800+ by placement)
  - Contest rating: ____ (Target: 1800+)
  - Easy: ____ | Medium: ____ | Hard: ____

Codeforces:
  - Rating: ____ (Target: 1400+)
  - Problems solved: ____ (Target: 300+)
  - Contests participated: ____

Projects:
  - GitHub repos: ____ (Target: 5+ quality repos)
  - GitHub stars: ____ (Target: 20+)
  - Open source PRs: ____ (Target: 5+)

Interviews:
  - Mocks completed: ____ (Target: 20+)
  - Real interviews given: ____
  - Offers received: ____
```

RESOURCES INDEX

Resource	Use For
Striver A2Z Sheet	DSA curriculum (primary)
CP-Algorithms.com	Algorithm deep dives
LeetCode Discuss	Company-specific interview experiences
Pramp.com	Free mock interviews
Gaurav Sen (YouTube)	System Design
ByteByteGo (Alex Xu)	System Design visuals
NeetCode.io	DSA patterns + roadmap
InterviewBit	Mock interviews + company problems
IndiaBix	Aptitude + Logical Reasoning
CS50 (YouTube)	OS, DBMS fundamentals
GFG	CS fundamentals articles
Designing Data-Intensive Apps (book)	Advanced distributed systems

---

FINAL NOTE

You're starting at LeetCode 53 on June 25, 2026.

If you follow this roadmap exactly — 5-7 hours daily, no excuses — you will be placement-ready by June 2027.

The 53 problems you've solved are not the peak. They're the foundation.

The grind is 365 days long. Every single day counts. Missing one day sets you back three.

```
June 25, 2026: 53 problems, Unrated.
July 25, 2026: 120 problems.
August 25, 2026: 220 problems.
September 25, 2026: 320 problems, CF Pupil.
December 25, 2026: A2Z complete, 500+ problems.
March 25, 2027: Projects done, resume ready.
June 25, 2027: Placement ready, 800+ problems.
```

Show up. Every. Single. Day.